Notes (Week 2 Monday)

These are the notes I wrote during the lecture.

```
Sets are uniquely defined by their *membership*

x ∈ S  pronounced "x in S" means that x is an element of S (member of)

  a ∈ {a,b,c}    b ∈ {a,b,c}    c ∈ {a,b,c}

  d ∉ {a,b,c}     (not a member of)

Two sets S and T are equal iff

  for all x,
    x ∈ S  if and only if   x ∈ T  (extensionality)

As a consequence:

  {1}      {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}
   are the same sets, because they have the same members.
   i.e. multiplicity doesn't matter


  S = {1,2,{3,4}}  how many elements? A: 3

  3 ∉ S
  {3,4} ∈ S      3 ∈ {3,4}

  set membership is not transitive: we count only the
   "top-level" elements

Last week, we looked unrestricted comprehension:

  {x: P(x)}   the set of all x such that P(x) holds

now our comprehensions look like this:

  {x: x ∈ 𝓤 and P(x)}
𝓤 (that's just U with a fancy font "calligraphic U")
 is pronounced "universe".
the universe is, intuitively, the set of everything
currently under consideration.

Why this extra complexity? A: Russell's paradox

Using unrestricted comprehension, we were able to write

  Y = {x : x ∉ x}, which is paradoxical:

  Y ∈ Y   ifff Y ∉ Y

Here, the issue is that unrestricted comprehension
 lets us conjure sets out of thin air.

  S = {x: x ∈ 𝓤 and P(x)}
  ^ forces the elements of S to be carved out of some
    pre-existing set 𝓤, meaning that we can no longer
    use comprehension to make "new" sets, only carve
```

```
    out specific subsets of old sets.

  {} : the most boring set
    x ∉ {}  for all x

  {{}}  "the set containing the empty set"
  - it has an element, and the element is empty
  {}
  - has no element

There is only one empty set.

Proof:
  Suppose there are two empty sets E1 and E2,
  and they are different.
  By extensionality, they must thus disagree about
  membership of an element. If so, one of them is not empty.


{x : x ∈ {1,2,3,4} and x is even} = {2,4}

Sometimes, we will still be sloppy and elide the x ∈ 𝓤 clause.
But it's still there in invisible ink.

A ⊆ B "A is a subset of B"
it means that every element of A is an element of B

{1,2} ⊆ {1,2,3}

Power set of X
  Pow(X)   𝓟(X)   the set of all subsets of X
  𝓟({{}}) = {{},{{}}}
  {} ⊆ {{}}        {} ⊆ A for all A
  {{}} ⊆ {{}}      A ⊆ A  for all A

Empty set is sometimes written: ∅

Q: Is it a problem that we define the powerset without
   having having a universe?
A: No.
   Usually, we'd assume that our universe is closed under
   certain standard operators (powerset, products)

We've seen (in the slides) that membership and subset are distinctL

  1 ∈ {1,2}    1 ⊈ {1,2}

Q: can you think an example where

  x ∈ S         x ⊆ S
A:  {} ∈ {{}}    {} ⊆ {{}}


  |X|  "the cardinality of X"
  card(X)    the number of elements in X

 |{1,2,3}| = 3
 |{1,1,1}| = 1

This is straightforward for finite sets, but gets weird for infinite
sets.

  |ℕ| = ??  there's no natural number we can put on the RHS
```

```
*If* we cared about the size of non-finite sets, we'd now
 introduce cardinal numbers, but we won't.

  |ℕ| < |ℝ|


 {0,1} ∪ {1,2,3} = {0,1,2,3}
 {0,1} ∩ {1,2,3} = {1}

A \ B   sometimes A - B    "the difference of A and B"
  everything that is in A, but not in B

  {0,1} - {1,2,3} = {0}

A ⊕ B  "the symmetric difference of A and B"
  everything that is in exactly one of A and B

  {0,1} ⊕ {1,2,3} = {0,2,3}

A^C = 𝓤 - A    "everything that is outside A"

but supposing the universe is ℕ

 (the even non-negative numbers)^C = (the odd non-negative numbers)

Q: can we use this notation for complementation

   Ā

A: Yes. We'll use this notation some in the course,
   but usually not for set complementation but for
   boolean negation

     Ā = ¬A

Q: In proofs, do we have to go from LHS to RHS?
A: No.


Let's prove idempotence A ∩ A = A

  A ∩ A = (identity)
  (A ∩ A) ∪ ∅ = (complementation)
  (A ∩ A) ∪ (A ∩ A^C) = (distributivity)
  A ∩ (A ∪ A^C) = (compl)
  A ∩ 𝓤 = (identity)
  A

The dual A ∩ A  is  A ∪ A
The dual of A is A

The principle of duality is:
  If A = B is provable from the
   laws of set operations
  then dual(A) = dual(B) also holds.

By invoking the principle of duality,
  we get from our proof above that A ∪ A = A

If you take a proof of A = A,
 and dualise all formulas in the proof,
 you get a proof of dual(A) = dual(A)
```

```
  A ∪ A = (identity)
  (A ∪ A) ∩ 𝓤 = (complementation)
  (A ∪ A) ∩ (A ∪ A^C) = (distributivity)
  A ∪ (A ∩ A^C) = (compl)
  A ∪ ∅ = (identity)
  A


 S × T  "the (Cartesian) product of S and T"
 is the of all pairs (x,y) such that
   x ∈ S and y ∈ T
 *ordered* pairs: (x,y) ≠ (y,x) (unless x=y)

 classic example: complex numbers
   ℝ × ℝ  coordinates

  (x,y) ∈ ∅ × S
    it must be the case that
      x ∈ ∅ and y ∈ S

  ∅ × S = ∅
   reminiscent of    0·x = 0

Formal languages are sets of strings

Strings are sequences of symbols,
 symbols are taken from an *alphabet*

Σ denotes the alphabet under consideration

empty word: λ or ε is: a string consisting of
  no alphabet symbols (the empty string)

 {} the empty language (a language with no words)
 ≠ {λ} the language containing the empty string

 vw   if v and w denote words, is the
      concatenation of v and w,
      all the symbols of v followed by all the
      symbols of w
 v·w

 aa·bb = aabb

 vwx = (vw)x = v(wx)  (concatenation is associative)

Σ^0  is all words of length 0 formed by symbols in Σ

Σ^0 = {λ}

Σ^1 = Σ

Σ = {0,1}

Σ^2 = {00,01,10,11}
...

Σ^*  (sigma-star)
 the set of all (finite) words over Σ

Σ^+ (sigma-plus)
```

```
  the set of all finite, non-empty words over Σ

A *language* is a set of strings over some Σ.
A *language* is a subset of Σ*

The set of all grammatically correct English sentences is a language.

The set of all syntactically correct C programs.

  0-1-1-2.2....

  -...0.

  -23523.1

The last clause prevents:
  -
  λ

Alphabets don't have to be over single characters;
  they can be over bigger things

In parsing, we'd call this sort of composite symbol
 a *token*
Generally, the first thing a compiler does when given a blob of text is to convert it to a blob of tokens.
When this fails, you typically get a lexical error in your face.

 baguette ∈ french ∩ english
```